

Problem A. Análise de Corridas

Neste problema, queremos contar o número de strings de tamanho n que possuem pelo menos uma ocorrência da substring dada. Para isso utilizaremos o algoritmo Knuth-Morris-Pratt (KMP), e o autômato construído a partir dele, com complexidade $O(m)$.

Caso n fosse pequeno, poderíamos resolver esse problema utilizando programação dinâmica, calculando, para cada $1 \leq i \leq n$, de quantas formas conseguimos chegar no estado $0 \leq j \leq m$ do autômato (representando uma string com os j últimos caracteres iguais aos j primeiros de s). Essa solução possui $n \cdot m$ estados com transição do tamanho do alfabeto, resultando na complexidade de tempo $O(26 \cdot n \cdot m)$, que é muito lenta para este problema, já que n pode chegar até 10^{18} .

Portanto, precisamos de uma forma mais rápida de resolver esse problema, que pode ser alcançada ao visualizar o autômato do KMP como um grafo. Nesse grafo, teremos todos os estados do autômato como nós, e as transições como arestas. Além disso, para garantir que não contaremos strings repetidas ou deixaremos de contar strings que não terminam no estado de aceitação (mas passaram por esse estado pelo menos uma vez), adicionamos um novo nó, que guardará todas as strings que passaram pelo menos uma vez pelo estado de aceitação. Com isso, o estado de aceitação deve ter uma aresta para esse estado final, e, para contar todas as possibilidades de strings que passaram uma vez por esse estado, o estado final deve ter 26 arestas direcionadas para si mesmo, uma para cada letra do alfabeto.

Seja A a matriz de adjacência do grafo montado. Sabemos que A^k conterá o número de caminhos de tamanho k entre dois vértices quaisquer. A partir disso, podemos observar que a resposta do problema será o número de caminhos de tamanho $n + 1$ do estado inicial do autômato até o nosso estado final, já que devemos considerar todas as letras utilizadas na string e também o passo adicional entre o estado de aceitação e o estado final. Podemos calcular a matriz A^{n+1} utilizando o algoritmo de exponenciação rápida com multiplicação de matrizes, resultando em uma complexidade final de $O(m + m^3 \cdot \log(n))$, a qual é suficiente para os limites deste problema.

Problem B. Bauru

Caso AB tenha 2 ou 4 dígitos, certamente a resposta é única: a primeira metade é A e a segunda é B . A razão para isso é que os dois números têm de 1 a 2 dígitos.

O caso mais complicado é quando AB tem três dígitos: XYZ . Pode-se verificar manualmente os casos $XY - Z$ e $X - YZ$ verificando que os números estão entre 1 e 99 e não são escritos com zeros à esquerda. Se ambos atenderem às condições, imprime-se -1 , caso contrário imprime-se a separação que atende ao enunciado.

Problem C. Cada K

Este problema pode ser resolvido com programação dinâmica (DP).

Podemos calcular a melhor resposta para todos os prefixos da lista; a resposta para o prefixo i será dp_i . Para o primeiro prefixo: $dp_1 = \max(A_1, 0)$. Depois, iteramos de 2 a N fazendo $dp_i = \max(A_i + dp_{i-k}, dp_{i-1})$ (se $i \leq k$, considere dp_{i-k} como 0).

Isso representa que a melhor resposta para o prefixo i é o melhor entre escolher a música i , usando a melhor resposta até k posições atrás, e não escolhê-la, simplesmente mantendo a melhor resposta calculada até o prefixo anterior. Isso funciona porque a melhor resposta só pode aumentar ao adicionar uma nova música ao prefixo, por isso podemos apenas olhar a melhor resposta calculada k posições atrás ou a última calculada.

Os limites da questão foram selecionados de forma que DPs de estados N e K também passem no limite de tempo.

Problem D. Djgsb ef Tijgu

Imagine que a frase favorita de Arthur é $[1, 3, 2]$. É possível shiftar a mensagem $[4, 5, 6]$ de modo que a frase de Arthur apareça? Não é possível. Analisando as versões shiftadas de $[4, 5, 6]$, é possível ver que as

diferenças entre os valores adjacentes se mantem constantes. O segundo valor sempre será 1 a mais que o primeiro, e o terceiro sempre será 1 a mais do que o segundo.

Então a frase [1, 3, 2] só poderia aparecer em uma seção da mensagem que tivesse as mesmas diferenças entre valores adjacentes, ou seja, o segundo valor é 2 a mais do que o primeiro e o terceiro 1 a menos do que o segundo.

Então podemos tomar um vetor DB, correspondente às diferenças adjacentes na frase B, e procurar por ocorrências dele em DA, um vetor correspondente às diferenças adjacentes na mensagem A. Podemos fazer isso com qualquer algoritmo de string matching, como Z-Function, KMP ou Hashing Polinomial.

Para cada match detectado, devemos descobrir qual foi o shift que permitiria essa ocorrência. Para isso, encontramos o carácter em A correspondente ao começo desse match e subtraímos desse valor o primeiro carácter de B. Podemos manter um vetor que contém o número de ocorrências de cada shift para ao final descobrirmos a maior.

Problem E. Estrutura de Dados Mais Fraca

A quantidade total de operações é no máximo 100, então implementações quadráticas também devem passar no limite de tempo e de memória.

O problema pode ser implementado usando uma lista de listas, inserindo elementos ao final delas de acordo com o enunciado. Na operação tipo 1, adiciona-se uma lista vazia à lista de listas, e na tipo 2, um número inteiro x à lista i . Guardando todas as listas, atender à operação do tipo 3 torna-se simples: basta acessar o j -ésimo elemento da lista i .

Pelos limites do problema, pode-se simular essa lista de listas até mesmo fixando que ela terá 100 listas de tamanho 100 cada uma, inicialmente preenchidas com zeros. A operação do tipo 1 pode ser completamente ignorada, e para a tipo 2 basta mudar o valor da primeira posição preenchida com um zero da lista i para x .

Problem F. Falatro

Para cada vértice s , o algoritmo realiza uma BFS para cada vizinho u no grafo sem s . Em seguida, o menor ciclo que contém u e s é dado pela menor distância até algum vizinho v de s , em que $v \neq u$. Assim, o menor ciclo que contém s é o menor ciclo obtido pelo algoritmo descrito.

1. Número de vizinhos: $O(deg(s))$
2. Cada BFS: $O(N + M)$

Portanto, o custo por vértice s é $O(deg(s) \cdot (N + M))$, e a complexidade fica:

$$O\left(\sum_{s=1}^N deg(s) \cdot (N + M)\right)$$

Problem G. Gelatos do Goiás

Para resolver este problema, precisamos maximizar a função de qualidade de um gelato, que é definida como $\min_{x \in E} (x) \cdot |E|$, obedecendo às restrições dadas pela permutação caótica.

A partir da permutação podemos descrever um grafo onde as arestas indicam que os ingredientes representados pelos vértices não podem ser utilizados em conjunto na receita. Com isso, temos algumas propriedades importantes do grafo:

- Cada vértice tem grau 2.
- O grafo é formado por ciclos disjuntos.

- Em um ciclo de tamanho $|C|$, o número máximo de vértices que podemos selecionar sem violar as restrições (ou seja, sem escolher dois vértices adjacentes) é $\lfloor \frac{|C|}{2} \rfloor$.

A ideia principal é fixar, em ordem decrescente, o ingrediente de menor valor no subconjunto selecionado. Para cada ingrediente i (com valor a_i), consideramos apenas os ingredientes com valores maiores ou iguais a a_i , com isso obtemos um subgrafo composto apenas por esses vértices e desconsiderando os demais, já que eles tem pontuação menor do que o ingrediente fixado.

Assim, à medida que processamos os ingredientes em ordem decrescente, vamos adicionando arestas ao grafo (quando ambos os extremos têm valores $\geq a_i$) e mantemos o estado dos componentes conexos usando uma **DSU**. Dessa forma, conseguimos manter, para cada componente, o tamanho e o número máximo de vértices que podem ser selecionados nele.

Enquanto estamos testando algum ingrediente, podemos visualizar como se estivéssemos adicionando o vértice que representa ele e as arestas que conectam ele e os elementos com pontuação maiores ou iguais a dele. Portanto, ao adicionar um ingrediente i (com valor a_i), temos três casos:

- O ingrediente não adiciona nenhuma aresta: Isso ocorre quando os dois ingredientes com os quais i é incompatível têm valores menores que a_i . Nesse caso, i forma um novo componente de tamanho 1. Adicionamos 1 ao total de vértices que estamos selecionando.
- O ingrediente adiciona uma aresta: Isso ocorre quando apenas um dos ingredientes incompatíveis tem valor $\geq a_i$. Então, i é conectado a um componente existente, aumentando seu tamanho em 1. Se o tamanho era s , agora é $s + 1$. Assim, o número máximo de vértices selecionados nesse componente é alterado. A contribuição do componente muda de $\lfloor \frac{s}{2} \rfloor$ para $\lceil \frac{s+1}{2} \rceil$.
- O ingrediente adiciona duas arestas: Isso ocorre quando ambos os ingredientes incompatíveis têm valores $\geq a_i$. Aqui, temos duas possibilidades:
 - Fechar um ciclo: Se os dois vértices incompatíveis já estão no mesmo componente, então adicionar i completa um ciclo. O número máximo de vértices selecionados nesse componente se torna $\lfloor \frac{|C|}{2} \rfloor$, onde $|C|$ é o tamanho do ciclo formado.
 - Unir dois componentes: Se os dois vértices estão em componentes diferentes, então unimos esses dois componentes, de tamanhos s_1 e s_2 . Com isso, devemos tratar primeiro do nosso teste: já que estamos fixando um ingrediente como o menor, devemos selecionar o vértice que o representa. Dessa forma, se algum dos componentes que estamos unindo tem tamanho ímpar, então neste teste a união deles contribuirá com $\lfloor \frac{(s_1+s_2+1)}{2} \rfloor$ selecionados. Caso contrário, $\lceil \frac{(s_1+s_2+1)}{2} \rceil$ serão selecionados. Por fim, o número máximo de vértices selecionados do novo componente (em testes futuros) passa a ser $\lceil \frac{(s_1+s_2+1)}{2} \rceil$, em que s_1 e s_2 são os tamanhos dos componentes originais.

Para evitar recálculo, manteremos uma variável ao longo desses testes que representa a soma do número máximo de vértices selecionados em todos os componentes no subgrafo atual, que é atualizada conforme os casos apresentados. Com isso, alcançamos uma complexidade total de $O(N \cdot \log(N))$, resultante da ordenação para processar os ingredientes de maior pontuação primeiro.

Problem H. Hardcore Aura Farming

Pode-se responder online as queries desse problema com o auxílio de uma árvore de segmentos (segtree) persistente, esparsa, e com propagação preguiçosa (lazy propagation).

Inicializemos uma segtree de soma apenas com zeros. Podemos enraizar a árvore dada em um vértice qualquer e aplicar na segtree o update associado a esse vértice. A partir dessa raiz, iniciamos uma DFS onde cada vértice copiará a segtree de seu pai e aplicará seu próprio update nessa cópia. Dessa maneira, podemos responder qualquer query de um vértice até a raiz. Para responder uma query entre dois vértices

S e F quaisquer, basta calcular o LCA W_q de S_q e F_q e responder $query(S_q) + query(F_q) - 2 \cdot query(W_q) + t$ onde $t = 0$ se $max(L_w, X_q) > min(R_w, Y_q)$, se não $t = (max(L_w, X_q) - min(R_w, Y_q) + 1) \cdot K_w$.

O LCA pode ser pré-computado em $O(N \log N)$ e respondido em $O(1)$, e cada update ou query custa $\log M$ em tempo, logo a solução será $O(N \log N + N \log M + Q \log M)$.

Problem I. Ivo viu o UVa

A solução é simples e pode ser feita de diversas formas. Uma delas é andar por cada letra na matriz e, se for 'v' ou 'V', verificar se há vizinhas nas 4 direções indicadas (horizontal, vertical e diagonais). Havendo vizinhos, basta conferir se uma delas é 'a' ou 'A' e se a outra é 'u' ou 'U'. Se for o caso, incrementa-se um contador. Atenção! Na entrada a quantidade de letras de cada linha é dada antes da quantidade de linhas.

Problem J. Jogo no Grid

A quantidade total de células é $N \cdot M$. Sabe-se que, em cada turno, uma das células é marcada. Portanto, o jogo terá exatamente $N \cdot M$ turnos com possibilidade de jogadas, e o jogador que realizar a $N \cdot M$ -ésima jogada será certamente o vencedor. Portanto, basta ver a paridade de $N \cdot M$.

Problem K. Kronos

1) Resumo da ideia:

- Ordene o vetor de timestamps t .
- Para cada consulta $[L, R]$, a resposta é: $idxR - idxL$, onde:
 - $idxL$ = primeira posição com valor $\geq L \rightarrow lower_bound(t.begin(), t.end(), L)$
 - $idxR$ = primeira posição com valor $> R \rightarrow upper_bound(t.begin(), t.end(), R)$
- O resultado $(idxR - idxL)$ é exatamente a quantidade de elementos t_i em $[L, R]$.

2) Porque funciona:

- $lower_bound$ devolve um iterador para o primeiro elemento não menor que L (isto é, $\geq L$).
- $upper_bound$ devolve um iterador para o primeiro elemento estritamente maior que R ($> R$).
- Todos os elementos entre essas duas posições (inclusive à esquerda, exclusivo à direita) são aqueles tais que $L \leq t_i \leq R$.

3) Complexidade:

- Ordenação: $O(N \log N)$
- Cada consulta: $O(\log N)$ para $lower_bound$ + $O(\log N)$ para $upper_bound \rightarrow O(\log N)$
- Total: $O(N \log N + Q \log N)$, adequado para N, Q até $2 \cdot 10^5$.

Problem L. Lá em cima

Podemos resumir o problema a encontrar a soma do quadrado dos $N + 1$ primeiros números ímpares e descartar 1 unidade. Por isso, aqui vamos focar apenas em achar essa soma. Há algumas formas de se fazer isso:

- A primeira e mais direta é usando a fórmula fechada: $\frac{N(4N^2-1)}{3}$
- Podemos reescrever a soma como $\sum_{i=0}^N (2i+1)^2 = \sum_{i=0}^N (4i^2 + 4i + 1)$

E calcular cada termo separadamente da seguinte forma:

$$\sum_{i=0}^N 4i^2 = 4 \cdot \sum_{i=0}^N i^2 = 4 \cdot \frac{N \cdot (N+1) \cdot (2N+1)}{6}$$

$$\sum_{i=0}^N 4i = 4 \cdot \frac{N \cdot (N+1)}{2}$$

$$\sum_{i=0}^N 1 = N$$

Assim, basta fazer as operações e somar os resultados.

iii) Uma última solução seria enxergar que existe um polinômio que descreve a formula, calcular alguns pontos dele e fazer uma interpolação de Lagrange para calcular no ponto N . Note que, de fato, o polinômio dado na primeira solução possui grau pequeno.

Importante: Como a resposta do problema é um número muito grande, não temos como calculá-lo e depois imprimir a resposta módulo $10^9 + 7$. Por isso, temos que ir calculando esse módulo a cada etapa das operações. Especificamente para a divisão é necessário calcular o inverso multiplicativo do número, não bastando fazer a divisão diretamente. Mais detalhes sobre isso podem ser encontrados nesse blog <https://codeforces.com/blog/entry/72527>

Problem M. Mapeamento Tático

Como os polígonos do plano são distintos e não se intersectam, eles formam uma árvore, onde o pai de um polígono é o menor polígono que o contenha estritamente (caso não seja contido por ninguém, o pai é a raiz da árvore, um vértice auxiliar). Considerando os pontos das perguntas como polígonos de um vértice só, basta ver a altura deles nessa árvore para contar quantos polígonos os contém.

Para construir essa árvore, pode-se usar a técnica Sweep Line. Nesse Sweep Line, irão ser processados os pontos das perguntas e as arestas dos polígonos que não sejam alinhadas com o eixo vertical. Cada ponto será um evento do Sweep Line, e cada aresta irá gerar um evento de criação (na extremidade mais à esquerda) e de remoção (na extremidade mais à direita). A ordem dos eventos é: coordenada X , adição depois remoção, coordenada Y (pontos das perguntas são considerados adições).

Durante o algoritmo, iremos manter a coordenada X que está sendo processada atualmente, chamada de X_a . Além disso, iremos manter um *set* que mantém os segmentos que foram adicionados mas ainda não removidos de forma ordenada. A ordem dos segmentos dentro do *set* é a seguinte: o segmento A vem antes do B se a intersecção entre A e a reta $x = X_a$ está abaixo da intersecção entre B e $x = X_a$.

Para fazer essa ordenação, basta manter o X_a como variável global e criar uma função que implementa essa comparação entre os segmentos (possível de calcular em $O(1)$) para ser passada ao *set*. Pode-se provar que essa ordenação é coerente desde que:

1. Os segmentos não tenham intersecção entre si (garantido pelo enunciado)
2. Os segmentos fiquem no *set* apenas enquanto existe intersecção entre eles e X_a (garantido pela forma de processamento do Sweep Line).

Assim, os eventos do Sweep Line são processados em ordem. Eventos de remoção apenas removem o segmento do *set*. Eventos de adição adicionam o segmento no *set* (se não se tratar de um ponto) e também calculam o pai do polígono ou ponto atual, caso ainda não tenha-se calculado o pai ainda. Para isso, basta ver o primeiro segmento do *set* cuja intersecção com X_a é maior do que a intersecção do atual com X_a (ou seja, o primeiro segmento acima do atual). Isso pode ser feito com busca binária, já que o *set* é ordenado. Caso a busca binária não encontre nenhum segmento mais acima, então o pai do atual é a raiz da árvore. Caso contrário, vemos se o segmento encontrado faz parte da borda superior ou inferior do polígono do qual faz parte — dizemos que um segmento faz parte da borda superior se e somente se ele liga $P1$ a $P2$ (considerando a ordem anti-horária) e $P2$ está à esquerda de $P1$. Se fizer parte da superior, então o pai do atual é ele (pois ele contém o atual), se não, o pai do atual é o pai dele (pois ele seria um irmão do atual na árvore).

Vemos que a árvore pode ser construída em $O(P \log P)$, onde P é o total de pontos da entrada. Em seguida, pode-se calcular a altura de cada vértice usando busca em largura ou profundidade. Por fim, basta imprimir as alturas dos vértices que são pontos das perguntas.

Problem N. Névoa

O problema pode ser resolvido por meio da simulação da estratégia de deslocamento do professor John.

Para isso, faça $X \leftarrow 0$, $step \leftarrow 1$, $d \leftarrow D$, $i \leftarrow 0$ e siga os seguintes passos enquanto $|step| < d$:

1. $d \leftarrow d - |step|$ (atualiza a distância percorrida)
2. $X \leftarrow X + step$ (reposiciona o barco)
3. se i for par, inverta o sinal de $step$; caso contrário, $step \leftarrow 2 \times step$ (corrige a direção de navegação)
4. $i \leftarrow i + 1$

Ao final do processo faltaram apenas d metros para atingir Y , de modo que $X \leftarrow X + d \times \text{sig}(step)$, onde $\text{sig}(x)$ é a função sinal de x .

Contudo, é preciso atentar à possível inconsistência dos dados, que ocorre quando X corresponde a uma posição já visitada. Para verificar esta condição, basta manter dois ponteiros L e R , inicialmente iguais a zero. Em cada iteração do algoritmo proposto, basta atualizar L e R para que registrem o menores e os maiores valores de X , respectivamente. Deste modo, $[L, R]$ será o intervalo das posições já visitadas. Ao fim da rotina, se $X \in [L, R]$, os dados são inconsistentes.

Esta solução tem complexidade $O(\log D)$.